ABAP INTERVIEW PREPARATION

ABAP quick revision

Senior Cloud decisions first, followed by essential concepts and common traps

329 questions across 24 study segments. Reviewed 11 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Read online in GitHub through README.md, or download the repository and open START_HERE.html for offline recall cards. The PDF is a portable reading and printing edition.

Clean Core at a glance

Clean Core levels

Classify actual SAP dependencies and extension technologies

A	**Released SAP APIs and extension points**	No corresponding finding
B	**SAP-designated classic APIs and options**	Priority 3 / information
C	**Internal SAP objects with upgrade risk**	Priority 2 / warning
D	**Non-recommended objects or techniques**	Priority 1 / error

A-D describes Clean Core usage checks, not every ATC finding.

Verify the target edition, release, classification data and configured checks.

Use the correct scope

These levels describe extension dependency choices. The ATC mapping is for the configured Clean Core usage checks. Public edition and ABAP for Cloud Development do not become unrestricted simply because a classic option exists in another edition.

Senior ABAP Cloud architecture decisions

CLOUD-01 A new approval extension needs immediate document validation and an external supplier portal. Would you put everything on-stack or on BTP?

I would split only where the business and transaction boundaries justify it. Validation that must reject the core document belongs at a supported on-stack extension point when available. The supplier portal may fit side-by-side because its users, scaling, and release lifecycle differ. I would define released remote contracts and tolerate explicit asynchronous states between the two. The decision depends on available APIs, latency, identity, operational ownership, and failure recovery, not a blanket preference for BTP.

Remember Deploying side-by-side does not make synchronous cross-system validation atomic or automatically clean-core compliant.

CLOUD-02 A stakeholder says ABAP Cloud means moving the existing application to SAP BTP ABAP environment. How would you correct the proposal?

I would separate the development model from the runtime. ABAP Cloud combines restricted language versions, released dependencies, modern tooling, and programming models such as RAP; it is available in supported S/4HANA environments as well as SAP BTP ABAP environment. Moving code to BTP changes data locality, connectivity, operations, and available business APIs. I would first identify the required product capabilities and choose deployment deliberately, then assess the application's compatibility with that environment.

Remember Cloud-compatible syntax does not establish identical application content, API availability, or deployment portability.

CLOUD-04 Two teams want independent releases but share pricing helpers and persistence types. How would you structure packages and software components?

I would identify which shared capabilities can carry a stable contract and which require coordinated releases. Packages organize implementation and access boundaries; software components also define lifecycle and delivery dependencies. A stable pricing interface can belong to a reuse component, while private persistence types should remain with their owner. Where supported, component relations permit targeted internal access, but they are not a substitute for a compatibility commitment. I would avoid cycles and make import order explicit.

Remember More software components do not create independent lifecycles when their dependencies still require lockstep delivery.

CLOUD-18 What would you require before approving a senior team's ABAP Cloud architecture for production?

I would require evidence for the main business invariants and nonfunctional risks, not only a component diagram. That includes transaction ownership, authorization, release-compatible dependencies, realistic workload behavior, observability, recovery, data retention, and deployment sequencing. Each consequential assumption needs an owner and a validation result. I would permit proportionate residual risk with explicit operational controls, while distinguishing a prototype, a locally tested implementation, and a solution proven on the intended runtime.

Remember A clean syntax check, green ATC run, or attractive diagram is not an end-to-end production-readiness certificate.

ABAP Cloud: senior RAP transactions and behavior design

SRAP-01 A new approval app has custom persistence, while its posting step must reuse an existing transactional API. Which RAP implementation type would you choose?

I would separate approval persistence from the posting boundary before selecting an implementation type. Managed RAP is my starting point for new custom tables. Managed with unmanaged save fits when RAP can own interaction buffering but an eligible API must own persistence. Fully unmanaged is justified when interaction behavior itself must adapt an existing transactional model. I would inspect the API's buffering, numbering, locks, authorizations and commit behavior; the label BAPI alone does not establish compatibility.

Remember Choosing unmanaged merely because the application is complex, or assuming managed with unmanaged save removes RAP transaction rules.

SRAP-03 A developer puts ordinary business validation in SAVE because the final data is available there. What would you change?

I would move predictable business rejection into the supported early checks, after required derivations and before the point of no return. The late save phase is designed to persist an already consistent transaction, not to provide routine interactive validation. A reported error alone is not a general rollback mechanism. I would also avoid the opposite absolute: SAP documents failure-capable saver support for specific BAPI integration scenarios, which must be verified against the actual release and saver contract.

Remember Saying either 'all errors can be returned from SAVE' or 'no supported late-save failure mechanism exists anywhere'.

SRAP-04 One action updates two local RAP BOs and then calls a remote payment service. Can the whole action be atomic?

Two compatible local RAP BOs can participate in the same RAP-controlled SAP LUW, with the transaction owner controlling the final save. An ordinary remote HTTP call is outside that local atomic boundary. If payment succeeds and local save fails, rollback cannot undo the remote effect. I would model a durable pending state and reliable asynchronous delivery, or an explicit compensating process, and define retry, reconciliation and customer-visible status rather than promise distributed atomicity.

Remember Assuming that calling HTTP inside an action enrolls the remote system in the SAP database transaction.

SRAP-08 A posting API returns the final document number only when called. How would you integrate numbering without publishing incorrect keys?

I would assess the documented late-numbering integration pattern and maintain preliminary identities until the save sequence assigns final keys. The adjustment response must correlate the preliminary identity in %pre with the final %key; consumers must not assume an earlier key is final. SAP's BAPI guidance places a numbering create call in ADJUST_NUMBERS and describes a failure-capable saver for that scenario. I would verify availability, API release status and failure semantics before choosing that design.

Remember Using SELECT MAX plus one, dropping preliminary correlation, or assuming ordinary ADJUST_NUMBERS accepts recoverable business failures without a supported special contract.

SRAP-12 A request changes CompanyCode to an unauthorized value and also makes its dates inconsistent. Would you use prechecks or validations?

I would separate the incoming-change decision from final-state consistency. A supported precheck can inspect requested values before they enter the transactional buffer, for example to reject a disallowed target context. A validation protects the resulting business state at its defined trigger time, such as the valid date interval. Neither replaces the complete RAP authorization model. I would merge partial input with relevant existing state when necessary and avoid assuming that every field is present in the request.

Remember Treating prechecks as a global security bypass or believing a UI mandatory marker enforces all backend invariants.

SRAP-16 An older RAP BO works without strict mode. The team wants to release it for cloud consumption by adding strict(2). Is that sufficient?

No. Strict mode introduces modeling and implementation requirements; enabling it can expose previously implicit behavior and incompatible derived-type usage. I would assess base and projection definitions together, make supported behavior explicit and review locks, authorizations, ETags and composition associations. API release also needs an intentional stable consumption contract and eligible dependencies. A successful syntax check is necessary evidence, but it does not demonstrate consumer compatibility, secure operation or a released API lifecycle.

Remember Treating strict(2) as a clean-core grade or API-release switch, or enabling it only on one layer and ignoring resulting contract changes.

Senior CDS, query contracts and security

MODEL-01 A new consumption entity exposes orders outside the user's sales organization. The base CDS has DCL. What do you investigate?

Trace the actual read entry point and its access control, including associations and alternative services. A protected base entity does not automatically confer equivalent protection on every higher entity. Define or inherit the intended restrictions through supported DCL constructs and mappings for the consumed entity. Keep operation authorization separate. The fix must preserve both authorized records and deliberate exceptions, rather than hiding the problem with a UI filter.

Remember Do not add #CHECK and assume it creates a missing rule.

MODEL-02 An approver may update organization A. Can an update move an order from A to B using only existing-instance authorization?

Authorization on the current instance can establish permission for its existing state, but that does not automatically approve the proposed target values. Treat the incoming organization change as a separate policy decision. Use supported prechecks and business validation/operation checks at appropriate points, with explicit handling of create and update. Derivations must not silently introduce a target scope that the caller was never allowed to choose.

Remember Do not equate authorization for the current object with authorization for any future object state.

MODEL-03 A custom query returns only authorized rows but an unrestricted total count. Is it secure?

No. Authorization must constrain the logical result before both counting and paging. A custom query provider owns the supported request semantics and cannot assume database-managed protections automatically cover remote or manually assembled data. Build one consistent authorized predicate and use it for data and count. Do not load all remote records into memory merely to hide unauthorized rows after an unrestricted count or aggregation.

Remember Filtering only the displayed page leaves count, aggregates and existence checks exposed.

MODEL-04 A query-provider filter cannot be converted to simple ranges. A developer catches the exception and returns everything. What is the correction?

Preserve the requested Boolean semantics or reject the unsupported filter clearly. Range conversion is not a universal representation of arbitrary nested AND/OR expressions and functions. Use a supported expression representation or another safe translation strategy appropriate to the provider. Combine user filters with mandatory authorization predicates independently, so a parsing failure cannot remove either restriction. Validate dynamic identifiers and values through supported APIs.

Remember A swallowed filter exception must not become an unrestricted selection.

MODEL-14 A CDS unit test passes, but the same entity leaks data for a restricted user. What may the test be missing?

Inspect the test environment and access-control configuration. A test focused on relational logic with CDS doubles may intentionally isolate or disable access control; it is not automatically an authorization test. Enable the supported access-control testing mechanism with meaningful role data where available, and add real service tests under restricted identities. Keep query-correctness evidence separate from security evidence.

Remember A green database-double test does not prove deployed authorization.

Senior RAP testing, performance and production

OPS-01 How would you test a Submit action without replacing the business object you are supposed to verify?

Keep the Submit implementation and its relevant BO behavior real. Isolate true dependencies such as database tables, external service adapters or other BOs using the appropriate supported doubles. Drive the operation through a meaningful boundary, often EML, and assert state changes, failures and messages. A double of the very action under test would prove the configured double response rather than the business rule.

Remember Do not label a test of a mocked Submit response as a test of Submit.

OPS-05 How do you prove optimistic concurrency rather than merely calling the same update twice?

Use independent consumers or sessions that read the same initial version, then perform conflicting updates in a controlled order. Assert that the first permitted write succeeds and the stale second precondition is rejected without lost data. For draft, separately test the exclusive-lock and total-ETag situations. A sequential test that rereads after every update never exercises a stale writer.

Remember Two updates in one shared transaction are not a realistic two-user concurrency test.

OPS-11 A remote approval succeeded, but the RAP caller timed out. How should an operator recover?

Treat the outcome as uncertain until reconciled using stable correlation. Do not blindly resubmit a non-idempotent business action. Identify which system owns each committed effect and use the documented status query, idempotency contract or compensation process. Preserve a durable recovery record and route unresolved cases to an owner. A local rollback cannot erase a remote commit.

Remember A timeout is not evidence that the other system performed no work.

OPS-14 What belongs in a RAP release gate beyond a passing ATC run?

Combine static findings with contract and behavior tests, relevant authorization cases, concurrency and rollback checks, representative performance evidence, dependency/API compatibility and target configuration readiness. Scale the gate to the changed risks. ATC can identify configured code issues but does not prove all business invariants or real integration behavior. Record unexecuted checks explicitly rather than converting missing evidence into a pass.

Remember A universal checklist is not a substitute for checking the risks actually changed.

Senior Clean Core governance and tradeoffs

GOV-01 A project claims Level A because its frontend uses RAP. How would you assess the extension?

I would define the assessment scope and inspect the actual dependency and extension paths, including adapters and called custom code. Level A relies on released contracts and the relevant supported development approach; Level B uses classified classic APIs, Level C internal objects, and Level D non-recommended dependencies or techniques. RAP or a Fiori frontend does not determine the classification alone. I would record object-level evidence and avoid presenting a mixed solution as uniformly Level A.

Remember RAP, BTP deployment, a customer namespace, or a green UI test is not proof that every underlying dependency is Level A.

GOV-02 An API is released with C1. A team assumes that means Clean Core Level C and permission to expose it remotely. How do you respond?

I would separate the naming systems. C0 is the Extend release contract, C1 is Use System-Internally, and C2 is Use as Remote API; these are purposes and stability contracts, not Clean Core A-D categories. A C1 release must also have the appropriate visibility for its consumer. Remote exposure needs a supported remote contract and communication design. I would inspect the object's API state and documented use rather than infer permissions from the letter C.

Remember C0/C1/C2 and Clean Core A/B/C/D are different classifications; release for one purpose does not release every possible usage.

GOV-04 An urgent release needs an ATC exemption for an internal dependency. What makes the exception governable?

I would require a specific business need, exact finding and source scope, target edition, alternatives considered, risk owner, and compensating verification. The exception should have an expiry or review trigger and a funded remediation path, not an indefinite blanket waiver. An ATC exemption changes handling of a finding; it does not make unsupported syntax legal, change SAP's API contract, or establish business correctness. The release decision must explicitly accept the residual exposure.

Remember An approved exemption is not a reclassification to Level A and must not become a general permission to bypass ABAP Cloud restrictions.

GOV-10 The direct dependency scan is clean, but an upgrade still breaks the extension. What could the governance model be missing?

I would inspect indirect custom dependencies, dynamic calls, configuration-selected implementations, remote contracts, and data assumptions outside the scanned scope. A scan can also miss relevant objects if its package selection, check version, or classification data is incomplete. I would connect findings to a dependency graph and process-level tests, recording evidence limitations. Clean direct references are useful, but they do not establish that every runtime path or semantic assumption is protected against the target change.

Remember A green direct-reference check is not proof that the whole solution has no unsupported or semantically fragile dependencies.

GOV-18 An interview panel shows an A-D chart with ATC priorities. What would you clarify before treating it as a release gate?

I would explain the chart's intended check context, then ask which variant, check implementations, priorities, classification data, object scope, and target release produced the findings. A mapping used by an API-usage check is not a universal meaning of every ATC priority. Other findings can concern security, correctness, or performance. A clean-core gate should preserve that distinction and state what a pass proves, what exemptions exist, and which runtime evidence remains necessary.

Remember Do not interpret every priority 1 finding as Level D, every priority 2 as Level C, or absence of findings as proof of complete Level A coverage.

ABAP foundations

ABAP-01 How do you choose between standard, sorted, and hashed internal tables?

I choose from the access pattern. A standard table suits indexed access and sequential processing. A sorted table maintains key order and supports efficient key and range access. A hashed table suits repeated complete-key lookup and requires a unique primary key. I also consider write volume, memory, and secondary keys before choosing.

Remember A hashed primary key is unique; that does not mean every field value must be unique or that arbitrary partial-key searches are constant time.

ABAP-07 How do you explain an SAP LUW versus a database LUW?

A database LUW is the database transaction completed by a database commit or rollback. An SAP LUW groups a business change across application processing, potentially spanning multiple database LUWs in classic dialog processing. The application coordinates consistency using appropriate persistence, update mechanisms, and logical locks. The transaction owner decides when the business operation is complete.

Remember A database commit caused by a work-process boundary is not permission to split a business operation into independently committed fragments.

ABAP-08 Who should issue COMMIT WORK, and what does AND WAIT guarantee?

The transaction owner should commit, rather than a reusable helper committing unexpectedly. In classic update-task processing, COMMIT WORK triggers registered update processing. AND WAIT waits for high-priority update completion: sy-subrc is 0 on success and 4 if that update fails. Without AND WAIT, COMMIT WORK sets sy-subrc to 0 without reporting the asynchronous update outcome. Neither form guarantees that every asynchronous follow-up in the landscape has completed successfully.

Remember COMMIT WORK AND WAIT is not a distributed transaction guarantee and does not authorize commits inside RAP handlers.

OO design and Clean ABAP

OO-02 What can an ABAP interface contain, and when would you use one?

An ABAP interface defines a contract implemented by classes. It can declare methods and other components such as types, constants, attributes, and events; it is not limited to abstract methods. I normally keep behavioral interfaces narrow so callers depend on the capability they need and tests can supply a substitute implementation.

Remember An interface is not an abstract class, and a collection of similarly named methods alone does not establish a meaningful common contract.

OO-03 How do ABSTRACT and FINAL affect classes and methods?

An abstract class cannot be instantiated directly and may provide shared implementation. It does not have to contain an abstract method. An abstract method has no implementation there and must be implemented before a concrete subclass can be instantiated. FINAL prevents further subclassing for a class, or further redefinition for a method.

Remember Do not import another language's virtual keyword or method-overloading rules into ABAP; use ABAP's actual redefinition and signature rules.

OO-07 How do dependency injection and dependency inversion differ?

Dependency inversion is a design principle: business policy should depend on suitable abstractions rather than concrete infrastructure details. Dependency injection is a way to supply a collaborator from outside, commonly through a constructor. Together they make dependencies visible and replaceable, but injection alone does not guarantee a useful abstraction or a good responsibility split.

Remember Injecting a concrete class everywhere may aid construction while still leaving the business layer tightly coupled to infrastructure.

ABAP SQL, HANA, and performance

SQL-02 What is the empty-table trap in FOR ALL ENTRIES?

If the driver internal table is empty, FOR ALL ENTRIES can cause the WHERE condition to be ignored and read all rows in scope. I guard the selection with a nonempty check and define the empty-input result explicitly. I also consider duplicate driver values, result duplicate elimination, and whether a join expresses the requirement better.

Remember An empty FAE driver does not mean no rows; a separate filter in the same WHERE clause is not a safe replacement for the guard.

SQL-04 How can a LEFT OUTER JOIN accidentally behave like an inner join?

A left outer join preserves unmatched left rows and supplies NULL for the missing right side. A WHERE condition requiring a right-side value can reject those unmatched rows. If the condition belongs to matching, I put it in ON; if it is intended to filter the final result, WHERE may be correct.

Remember Moving conditions between ON and WHERE is not always semantics-preserving for outer joins.

SQL-10 How do ST05, SAT, SQL Monitor, and database plans complement each other?

ST05 helps inspect traced database access for a scoped execution. SAT shows ABAP runtime distribution and call behavior. SQL Monitor helps find important SQL across a broader workload. A database execution plan explains how the database processes a statement. I select the tool according to whether the bottleneck is database work, ABAP processing, or call volume.

Remember The slowest individual SQL statement may not be the largest total cost if another cheap statement is executed thousands of times.

CDS data modeling

CDS-03 Association or join and how do you select cardinality?

An association models a reusable relationship; a path expression can cause a join when the relationship is consumed. A join explicitly combines sources in the current query. Cardinality must describe the actual data relationship and ON condition. Wrong to-one assumptions can produce incorrect or optimizer-dependent results.

Remember Associations are not automatically faster than joins and do not enforce referential integrity.

CDS-07 Does DCL on a base view automatically protect every higher view?

No. Read authorization must be designed for each externally accessed CDS entity. A higher view can use its own access control with inherited conditions where supported. Merely adding AccessControl.authorizationCheck: #CHECK does not invent an access rule. RAP operation authorization is a separate concern.

Remember DCL on one layer does not automatically secure an arbitrary view-on-view stack.

CDS-11 How do CDS keys and database keys relate?

A CDS key communicates entity identity to consumers but does not automatically enforce uniqueness in the query result. Database keys enforce uniqueness in persistence. Joins, missing key elements and aggregations can change result uniqueness. RAP and OData need stable, truly unique entity identities.

Remember Marking an element key does not remove duplicate rows.

RAP foundations and transactions

RAP-02 How do you choose managed versus unmanaged RAP?

Managed RAP supplies the transactional buffer and standard persistence handling for a suitable data model. Unmanaged RAP lets existing business logic supply interaction and save implementations while still obeying RAP's transaction contract. Managed with unmanaged save keeps managed interaction but delegates persistence. Choose by ownership of behavior and persistence, not application age alone.

Remember Unmanaged does not permit arbitrary COMMIT WORK or database changes anywhere in a handler.

RAP-04 Where may COMMIT ENTITIES be called?

An external ABAP EML consumer can use COMMIT ENTITIES to trigger the save sequence. Inside a RAP behavior implementation, the framework owns commit and rollback. Do not call COMMIT ENTITIES or COMMIT WORK in actions, validations, determinations or saver methods; violating the contract can cause a runtime error.

Remember Moving the commit from validation into SAVE or CANCEL does not make it valid.

RAP-07 How do FAILED REPORTED and MAPPED differ?

FAILED identifies instances or operations that failed. REPORTED carries messages and their context, including severity and field references. MAPPED returns identity mappings where the operation contract supports them, for example creation. A message alone does not necessarily reject the operation.

Remember Error severity in REPORTED is not a substitute for FAILED; determinations do not expose FAILED.

RAP-10 What do %cid %cid_ref %tky and %pid mean?

%cid correlates a created instance within a request. %cid_ref refers to a related creation by that content ID. %tky is the transactional key, including the relevant identity context such as draft state. %pid is a preliminary identifier used in late-numbering scenarios. They have different lifetimes and purposes.

Remember %tky means transactional key, not temporary key; %cid is not a persisted business document number.

RAP-11 Early numbering versus late numbering?

Early numbering supplies the final key during interaction, either by the consumer or implementation as supported. Late numbering assigns final identifiers late in save, after preliminary identity has been used. Managed UUID numbering is a common technical-key option. Business numbering and technical identity may be separate requirements.

Remember Late numbering alone does not guarantee a gap-free sequence, and SELECT MAX plus one is unsafe under concurrency.

RAP-12 What problem does draft solve?

Draft persists an incomplete editing state across stateless requests and interruptions, separately from active business data. The framework provides a draft lifecycle with operations such as Edit, Activate, Discard and Resume; Prepare can run designated checks and determinations. Draft and active identities must be distinguished.

Remember Every BO is not automatically draft-enabled, and a custom Submit action is not automatically draft activation.

RAP service design and advanced behavior

ADV-01 How do pessimistic locks and ETags complement each other?

A pessimistic lock prevents conflicting processing for a time. An ETag detects whether the version a client read is still current when it writes. RAP defines lock ownership and optimistic concurrency through behavior. Draft scenarios also need total-ETag handling for relevant active-data changes.

Remember Locking and logging are different concepts; an ETag is not a log and does not reserve the object.

ADV-02 Global authorization instance authorization or feature control?

Global authorization checks permissions independent of a specific instance. Instance authorization considers an instance's context. Feature control enables operations or fields according to business state. UI availability and feature control help consumers, but authorization must enforce who may perform the operation.

Remember Hiding or disabling a button is not an authorization check.

ADV-06 What is augmentation and how does it differ from additional save?

Augmentation adapts supported modifying operations at a RAP projection boundary, allowing projection-specific input to be enriched or translated. Additional save adds persistence-related work alongside a managed save scenario. They run for different purposes and are not interchangeable hooks.

Remember Augmentation is not a generic before-save hook for copying every virtual element into a table.

ADV-13 How do static and dynamic feature controls differ?

Static restrictions are declared in the behavior definition, for example a readonly or mandatory field. Dynamic controls determine supported operation or field availability from global or instance context. They express business availability, while authorization remains separate. Check which controls the operation and target release support.

Remember A dynamic UI state does not prove security, and mandatory metadata does not replace all backend business validations.

OData Gateway and Fiori integration

ODATA-01 When would you choose OData V4 rather than V2?

Prefer V4 for new supported transactional RAP services, while checking the actual Fiori elements or integration consumer and backend release. Keep V2 where required by an existing contract or consumer. V4 has first-class actions and functions and different protocol details; it is not simply a faster V2 switch.

Remember Neither RAP nor draft automatically implies that V2 is impossible; Gateway is not exclusively V2.

ODATA-04 How do paging and total count work in V2 and V4?

Apply filtering before count and paging, and ensure a stable order. V2 commonly uses $inlinecount=allpages, while V4 uses $count=true for an inline total; the /$count resource returns a count. Server-driven paging can return a next link that clients should follow.

Remember Do not invent one generic SET_COUNT method for every Gateway implementation; use the actual response contract.

ODATA-07 Is a batch request one database transaction?

Not necessarily. A batch groups HTTP operations; it is not automatically one atomic transaction. Multipart batch formats use changesets for atomic change operations. OData 4.01 JSON batch uses atomicity groups. These mechanisms depend on protocol version and provider support; read operations and separate groups can have different outcomes. Deep create describes creating a related structure and requires backend support for a coherent transaction.

Remember HTTP 200 for a batch does not mean every operation inside it succeeded.

Clean Core levels and extensibility

CORE-02 What are Clean Core levels A B C and D?

A uses released SAP interfaces and extension points with their supported contracts. B uses SAP-designated classic APIs and supported classic extension options. C depends on internal SAP objects with upgrade risk. D uses explicitly non-recommended objects or techniques. Always classify actual dependencies and technology in the applicable product context.

Remember A familiar API is not automatically B, and every unreleased object is not automatically D.

CORE-03 How do the levels map to ATC messages?

In the configured Clean Core usage checks, A produces no corresponding finding, B is priority 3 information, C is priority 2 warning and D is priority 1 error. This is the Clean Core mapping, not a universal interpretation of every ATC result from security or performance checks.

Remember No ATC findings does not prove that the entire application is secure, correct or fully Level A.

CORE-08 Does wrapping an unreleased SAP API make it Level A?

A released custom wrapper can provide a controlled interface to a consumer where the platform permits it. Its implementation still has the classification and upgrade risk of the underlying dependency. Evaluate both consumer contract and implementation chain, and retire the wrapper when an appropriate supported alternative exists.

Remember A wrapper is not a mechanism for laundering an internal or prohibited dependency into an entirely Level A solution.

CORE-10 How do you choose key-user on-stack and side-by-side extensibility?

Choose key-user tools for supported business-context adaptations. Choose on-stack developer extensibility when close access to released data and transactional integration is required. Choose side-by-side for a more independent application lifecycle or cross-system process, using supported remote interfaces and explicit distributed consistency.

Remember These are placement and development choices, not automatic A-D grades or fixed implementation times.

CORE-15 What if no released API covers the business requirement?

Recheck standard functionality and supported extension points, confirm the target release, and evaluate another placement or process design. In a landscape allowing classic development, inspect designated classic APIs and govern any remaining internal dependency. Request a supported API through the appropriate SAP channel when needed.

Remember Do not immediately choose a modification, and do not assume public edition permits a classic workaround.

BTP and cloud integration for ABAP developers

BTP-04 What are a service instance binding and service key?

A service instance is a provisioned service resource. A binding connects an application to that instance and supplies connection material. A service key provides credentials for a supported external client use case. A binding does not universally mean that a separately managed service-key artifact was created.

Remember default-env.json is a local development convention, not the universal Cloud Foundry credential transport.

BTP-07 How do remote transactions differ from local RAP transactions?

A remote API call generally has a separate transaction boundary. A local rollback does not automatically undo a remote commit. Design orchestration with idempotency, explicit statuses, retries and compensation where needed. Do not assume distributed ACID behavior merely because both systems use SAP technology.

Remember Calling HTTP inside an action does not make the remote change part of the local RAP database transaction.

RICEFW, enhancements, and workflow

RICEFW-11 How should a RAP submit action start an approval process reliably?

The action should validate the requested transition and update the business object through its transaction contract. Starting an external process must be coordinated with successful persistence using supported event or integration mechanisms. I design correlation, retries, and idempotency so rollback does not leave an orphan approval and duplicate delivery does not create duplicate workflows.

Remember Calling an external workflow endpoint before save succeeds can start approval for a document that never committed.

RICEFW-15 Which S/4HANA changes can affect existing ABAP developments?

Check target-release simplification items and custom-code findings before changing the application. Relevant areas include inventory data modeling around MATDOC, the Universal Journal around ACDOCA, Business Partner and CVI, and application-specific output and transaction changes. Compatibility mechanisms and migration prerequisites vary; do not infer behavior from a table name alone.

Remember Replacing a legacy table name with MATDOC or ACDOCA mechanically is not a migration strategy.

Testing security and delivery

TEST-03 How would you test a RAP business object?

Use suitable EML-driven tests to exercise the real BO behavior with controlled persistence and dependencies. Cover allowed and rejected operations, response messages, save failures, authorization and concurrency where appropriate. Add service-level tests for protocol behavior and UI tests for user flows. Framework availability depends on the target release.

Remember A mock Fiori success or a doubled BO does not establish that the real RAP implementation works.

TEST-06 What security tests would you include?

Test both permitted and forbidden reads and operations with realistic user roles and organizational scopes. Check direct API calls, mass operations and hidden fields, not only the UI. Verify authentication, DCL and behavior authorization separately and inspect error messages for unintended data exposure.

Remember Feature control, UI.hidden and disabled buttons are not sufficient security boundaries.

HTTP, REST, JSON and XML

HTTP-01 What is the difference between HTTP, REST, OData and a payload format?

HTTP supplies request/response semantics. REST is an architectural style with constraints such as stateless interactions and a uniform interface. OData specifies conventions for data services. JSON and XML represent data; neither makes an endpoint RESTful.

Remember Do not use REST, HTTP and JSON as interchangeable names.

HTTP-04 What do safe and idempotent mean, and how do they affect retries?

Safe methods have read-oriented requested semantics. Idempotency concerns the intended effect of repeated identical requests, not identical response codes. PUT and DELETE are idempotent by their HTTP semantics; POST and PATCH need specific application guarantees for safe repetition.

Remember Do not blindly retry a potentially committed business creation because the response was lost.

HTTP-07 What must you preserve when mapping JSON to ABAP?

JSON supports objects, arrays, strings, numbers, booleans and null. Preserve the API contract for property names, missing versus null values, number precision and ordered array entries. Do not depend on object member ordering or duplicate member names.

Remember Do not coerce every missing or null property into the same initial ABAP value without considering update semantics.

HTTP-08 What XML concepts matter in an integration interview?

XML is a structured representation with elements, attributes and nested content. Well-formedness checks syntax; schema validation checks additional declared constraints. A namespace is identified by its namespace URI. An element or attribute's expanded name combines its namespace name, if any, with its local name; the chosen prefix spelling is only an alias.

Remember Do not process arbitrary XML with string splitting or assume that well-formed XML is business-valid.

Function modules, RFC, BAPI, parallel processing, and jobs

RFC-02 How do synchronous, asynchronous, transactional, queued, and background RFC differ?

Synchronous RFC waits for the response. Asynchronous RFC lets the caller continue and optionally collect results through callbacks. Transactional RFC registers calls and parameters in the current SAP LUW; COMMIT WORK triggers processing of the registered unit rather than each call running immediately. Queued RFC adds sequencing through configured queues. Background RFC supports transactional and queued units with its own infrastructure. Their documented delivery guarantees apply to those units, not automatically to an entire distributed business process.

Remember STARTING NEW TASK is not durable background messaging, and queued processing does not imply one global order across unrelated queues.

RFC-04 When should BAPI_TRANSACTION_COMMIT or BAPI_TRANSACTION_ROLLBACK be called, and what does WAIT mean?

The owner of a BAPI transaction decides whether to commit after evaluating the complete return contract or roll back after a failure. For remote calls, transaction control must reach the correct remote context. WAIT requests synchronous high-priority update completion; it does not wait for every downstream workflow, interface, or lower-priority activity. I verify exceptions in the specific BAPI documentation.

Remember An empty RETURN table is not the only possible success condition, and a local COMMIT WORK does not commit an unrelated remote BAPI context.

RFC-07 How do you prevent parallel RFC processing from exhausting resources?

In classic parallel processing I use DESTINATION IN GROUP with an appropriate RFC server group, bound the number of outstanding tasks, and size work packages deliberately. SPBT_INITIALIZE can help inspect available resources, but availability changes. I handle RESOURCE_FAILURE with bounded retry or throttling and measure whether parallelism actually helps the workload.

Remember A resource count is a snapshot, not a reservation; hard-coded maximum concurrency can overload the system or increase lock contention.

RFC-10 How do you schedule and troubleshoot a classic background job?

In a classic system I define job steps, program variants, execution identity, and start conditions, commonly through SM36. I monitor status, step details, job log, and spool through SM37, then correlate application logs, dumps, and external dependencies. Before repeating a failed job, I establish which business effects already committed and whether the input selection is stable.

Remember SM37 Finished is not proof that every business record succeeded, and canceling a job does not reverse its earlier commits.

IDoc, ALE, AIF, and interface operations

INTF-01 What does an IDoc contain, and how do message type, basic type, and extension differ?

An IDoc contains a control record identifying routing and message characteristics, data records holding segment payloads, and status records documenting processing. The message type expresses business meaning. The basic type defines the standard segment structure, while an extension adds supported customer segments. These concepts must agree with sender, receiver, and application processing capabilities.

Remember The message type is not the same as the basic type, and the IDoc document number is not automatically the target business document number.

INTF-07 What is SAP Application Interface Framework, and what is it not?

AIF provides configurable interface processing, monitoring, and error-handling capabilities so technical and business users can investigate messages in a meaningful context. Depending on the scenario, it can organize mappings, checks, actions, and recovery. It does not replace the underlying transport or automatically monitor every interface. Availability, delivered content, and custom-development usage rights must be verified.

Remember AIF is not a substitute for RFC, IDoc, OData, or middleware, and its presence does not establish unrestricted licensing or identical cloud capabilities.

INTF-11 When would you use OData, IDoc, or AIF?

OData is an HTTP-based service protocol useful for supported queries and business operations, including UI and system integration. IDoc provides structured messages and established asynchronous application integration scenarios. AIF adds processing and operational capabilities around supported interfaces. They address different concerns and can coexist; I choose the business contract and delivery model before selecting monitoring.

Remember AIF is not a third wire protocol, OData is not limited to UI use, and IDoc is not restricted to batch-only dispatch.

Practical ABAP: values, expressions, and failure paths

BASE-01 What changes when a method receives a CHANGING parameter by value instead of by reference?

By reference, the formal parameter aliases the caller's data, so allowed writes affect it immediately. With VALUE, a local value is changed and copied back on normal completion. An exception leaving the method prevents that normal copy-back. This difference affects error guarantees, but it does not undo database writes or changes made through referenced objects.

Remember Pass-by-value is not a transaction rollback and does not clone objects behind reference parameters.

BASE-04 A dynamic ASSIGN fails, but IS ASSIGNED is still true. How can the program update the wrong field?

A failed dynamic assignment can leave the field symbol pointing at its previous target. IS ASSIGNED then says only that some assignment exists, not that the latest lookup succeeded. I check the statement's result immediately and avoid using the alias after failure. On a supporting release, ELSE UNASSIGN makes the failure state explicit; otherwise I unassign before attempting the lookup.

Remember An assigned field symbol is not evidence that the most recent dynamic ASSIGN succeeded.

BASE-06 An import must reject precision loss rather than silently round or truncate. Where does EXACT help?

EXACT requests a lossless assignment or calculation and raises an appropriate exception when the requested result cannot be represented without loss. CONV applies ordinary conversion rules and is not equivalent validation. I place the check at a typed boundary, translate conversion failures into useful input feedback, and still validate domain rules such as permitted signs, identifiers, and currency.

Remember EXACT does not prove that a technically representable value is an allowed business value.

BASE-12 A report accepts a filter value and a sort-column name. Is escaping quotes sufficient for safe SQL?

No. Data values and SQL identifiers are different inputs. I bind values through host variables in static ABAP SQL where possible. For configurable identifiers or dynamic clauses, I restrict them to a server-controlled allowlist and validate the supported syntax. Escaping a string value cannot authorize a table name, column name, operator, or an entire user-supplied WHERE fragment.

Remember Input that is safely quoted as a literal is not therefore safe as SQL syntax or an authorized data access.

BASE-16 An authorization helper returns an empty range table. Why is WHERE field IN range unsafe as the only access restriction?

An initial selection range represents no restriction for IN, so it must not be used to encode 'the user may see nothing.' I define empty authorization results separately and return no rows or reject the operation before querying. Nonempty ranges combine inclusion and exclusion rules; their semantics differ from a plain list whose empty membership set would match nothing.

Remember An empty ABAP selection range is not the same as an SQL list containing zero allowed values.

Practical OO: contracts, state, and substitutability

OODEEP-01 A method named reserve returns a reservation ID. Which promises must be explicit beyond its parameter types?

I define accepted inputs, the observable success result, allowed state changes, and failure behavior. The caller needs to know whether the reservation is merely buffered or durable, whether repetition is safe, and who owns transaction completion. Those promises belong in the contract and its tests. A matching signature alone cannot tell another developer whether combining two operations is safe.

Remember RETURNING an identifier does not itself promise committed persistence, and a method name cannot replace a transaction contract.

OODEEP-09 A repository raises a low-level exception. Should a business service catch CX_ROOT and return an empty result?

Usually not: an empty result can turn technical failure into a false business fact. I catch failures at a boundary that can recover or translate them into the service's declared meaning. When wrapping an exception, I preserve the original through previous so diagnosis remains possible. Expected absence, business rejection, and technical failure need distinguishable outcomes.

Remember Broad catch-and-default handling is not resilience when it changes an unknown outcome into valid-looking data.

Practical ABAP design patterns and trade-offs

PAT-01 Approval rules vary by country, while the application workflow should remain stable. Strategy or Template Method?

Prefer an injected Strategy when approval calculation is the variation and the caller should select or replace it without changing its own class hierarchy. Template Method fits a deliberately stable algorithm skeleton with controlled subclass extension steps. Here, I would keep workflow coordination separate from country rules and pass explicit context to the selected rule. Inheritance becomes justified only if its protected extension contract and lifecycle are genuinely stable requirements.

Remember Choosing inheritance solely to share code, or allowing a rule implementation to take over transaction ownership.

PAT-03 A legacy posting API has awkward parameters and a three-step prepare/check/post sequence. Would you build an Adapter, a Facade, or both?

Use an Adapter to translate the application's posting contract into the legacy API's types, identities and error representation. A Facade can expose a coherent business operation over the three underlying steps. They may coexist, but their boundaries must preserve authorization, failure information and transaction ownership. Neither pattern makes an unreleased API eligible for ABAP Cloud use or removes a nested commit. I would reject a superficially clean wrapper that hides those constraints.

Remember Treating any wrapper as both a supported API and a transaction adapter without inspecting its dependencies.

PAT-08 A repository interface improves testability, but its RAP implementation reads managed persistence directly. What can go wrong?

The seam may be useful while the implementation is transactionally incorrect. A direct table read can miss creates, updates and deletes still in the RAP buffer. Define whether the repository contract means persisted state or the current transactional view; for transaction-sensitive BO logic, use the appropriate EML behavior access. Do not add a generic repository that writes managed tables and bypasses RAP checks. Test doubles must reproduce the relevant contract, not merely return convenient rows.

Remember Equating a mockable interface with correct behavior or recreating RAP's transactional buffer in a second general-purpose cache.

PAT-09 A singleton caches the current approver and makes tests pass alone but fail as a suite. How do you redesign its lifetime?

I would remove request-specific mutable state from the singleton and make dependencies explicit. A conventional ABAP singleton is scoped to its internal session, not to all users or the whole landscape, and its lifetime can exceed one test or business operation. Prefer a freshly constructed, context-aware service or a cache with explicit key, scope and invalidation. If a shared instance is justified, specify immutability or cleanup rules and provide deterministic substitution in tests.

Remember Claiming singleton means one system-wide instance or solving hidden state solely by forcing test order.

Supporting skills: HANA decisions and source-to-release delivery

SQLX-01 A CDS table function joins client-dependent purchasing data in SQLScript, writes a helper table, and assumes the outer CDS DCL protects every internal read. Would you approve it?

No. For the documented CDS table-function contract, the AMDP database function is read-only, uses SQLScript on HANA and must satisfy the applicable client-safety rules. I would remove the write and review every source, join and client boundary rather than trust an outer annotation. Client isolation and business authorization are separate concerns: establish protection at the supported consumption path and do not assume SQLScript reproduces ABAP SQL or CDS access-control behavior automatically. Confirm target-release and language-version eligibility first.

Remember Treating every AMDP method as read-only, or treating this read-only table-function constraint as permission to read any SAP table. General AMDP capabilities and eligible dependencies are separate.

DELIV-01 A pull request passes local linting and its ABAP sources are pushed with abapGit. The team calls the cloud feature deployed. What evidence and delivery steps are missing?

Git records source history; abapGit transfers supported development objects. Neither statement proves target activation or productive delivery. Establish the product-specific lifecycle route, software component and dependency order; BTP ABAP environment can use supported transport/gCTS or add-on delivery scenarios, while other ABAP Cloud environments have their own routes. Tie the source revision to target syntax/activation, ATC, meaningful ABAP Unit and integration results, then verify the imported revision, service access and configuration in the intended system.

Remember Equating an abapGit push with a transport release, treating gCTS and abapGit as interchangeable names, or applying one BTP delivery route to every ABAP Cloud product.